Circuit Solver

Project Plan—Iteration #1

Team 02

Jayden Tran
Joseph Gereige
Tianhong “T” Zhong

EECE 2140: Computing Fundamentals for Engineers
Instructor: Dr. Fatema
Submission Date: 10/13/2026
GitHub Repository: https://github.com/zhongtia111/circuit_solver_2140.git

1. Project Scope and Expected Deliverables

1.1 Project Description

Circuit Solver is a C++ console program that analyzes DC circuits made up of resistors, independent voltage sources, and independent current sources. The user describes the circuit as a netlist, either typed into the terminal or loaded from a text file, by listing each component, the two nodes it connects, and its value. The program builds a system of equations using nodal analysis, solves it with Gaussian elimination, and reports every node voltage, branch current, and the power absorbed or delivered by each component. The intended users are students in introductory circuits courses who want to check their hand calculations quickly. The program also has to catch invalid input, such as missing a ground node, negative resistance values, or a circuit that can't be solved.

1.2 In Scope

- Read a circuit netlist from a text file or from user input in the terminal
- Support resistors, independent DC voltage sources, and independent DC current sources
- Build and solve the nodal analysis equations to find all node voltages
- Calculate the current through and power for every component
- Show the results in a clearly formatted table in the terminal
- Check for input errors (bad formatting, missing ground node, invalid values) and give helpful error messages
- Test the program against at least 10 circuits with hand-calculated answers to confirm accuracy

1.3 Out of Scope

- AC, phasor, or time-domain (transient) analysis
- Capacitors, inductors, diodes, transistors, and op-amps
- Dependent (controlled) sources
- A graphical user interface or schematic editor
- Exporting results to files other than plain text

1.4 Expected Deliverables

#	Deliverable	Description	Due Date
1	Project plan (this document)	Defines project scope, individual and team objectives, selected tools, timeline with assigned responsibilities, contribution table, and meeting summaries.	10/13/26
2	Working source code	C++ console program in the team GitHub repository that reads a circuit netlist, solves it using nodal analysis, and outputs node	*TBD*

#	Deliverable	Description	Due Date
		voltages, branch currents, and component power.	
3	Test cases and results	Set of at least 10 DC circuits with hand-calculated answers, stored in the repository, along with a comparison showing the solver's output matches expected values.	TBD
4	Final documentation / README	README in the repository explaining how to compile and run the program, the netlist input format with an example, sample output, and known limitations.	TBD
5	Final presentation	Live demonstration of Circuit Solver solving example circuits, plus an overview of the design, testing process, and each member's contributions.	12/08/2026

2. Objectives

All objectives below are written to be clear, realistic, and measurable.

2.1 Team Objectives

#	Objective	How We Will Measure It	Target Date
1	Solver correctly computes node voltages for DC resistive circuits	Matches hand-calculated answers within 0.1% on 10 test circuits	11/17
2	Program rejects invalid netlists with clear error messages	Correct error shown for 5 invalid test inputs	11/17
3	Program compiles cleanly and is documented	Builds with g++ -std=c++17 -Wall -Wextra -pedantic with no warnings, and the README is complete	11/24

2.2 Individual Objectives

Jayden Tran

Objective	How I Will Contribute	How It Will Be Measured
Build the netlist input parser	Read components from a file or the terminal into the shared data structures, and check for missing ground, negative values, and bad formatting	Parses all 10 test netlists correctly, and shows the correct error message for each of the 5 invalid inputs
Validate input	Create hand-solved test circuits, run them through the program, and record the results	At least 10 test circuits with expected answers committed to the repository by 11/17, with all results documented

Joseph Gereige

Objective	How I Will Contribute	How It Will Be Measured
Build the nodal analysis solver	Write the nodal analysis equations and Gaussian elimination	Node voltages match hand calculations within 0.1% on all 10 test circuits
Handle unsolvable circuits	Detect singular matrices and report an error	Error shown for every unsolvable test circuit

Tianhong Zhong

Objective	How I Will Contribute	How It Will Be Measured
Set up and manage Git repo	Set up the github repository, manage codebase, resolve issues	Be able to have everyone access the github repo and make commits and PRs
Handle input validation	Ensure inputs to system are valid, make system response robust	Test with a variety of possible inputs, ensure that they are valid and if not provide response

3. Technologies and Tools

Technology / Tool	Purpose	Why We Chose It
C++	Main programming language	Language used in EECE 2140; fast, and all members are already familiar with it
VS Code	Code editor	Free, cross-platform, integrated terminal and Git support
Git and GitHub	Version control and collaboration	Tracks every change, lets members work on separate branches, required for submission

Technology / Tool	Purpose	Why We Chose It
Compiler g++	Building the program	Course standard for c++ compiler
Instagram Group Chat	Team communication	Accessible amongst team

4. Project Timeline

4.1 Tasks, Responsibilities, Milestones, and Deadlines

Milestone	Task	Owner	Deadline	Deliverable	Status
M1: Planning	Write project plan and set up GitHub repository	all	10/13	Project plan PDF; repository	*[ ]*
M2: Design	Define circuit input format and data structures	T	10/18	Design notes in repo	*[ ]*
M2: Design	Choose solving method, e.g., nodal analysis	Joseph	10/20	Written method summary	*[ ]*
M3: Build	Input parsing	T	11/03	Parser code	*[ ]*
M3: Build	Matrix setup and solver	Joseph	11/10	Solver code	*[ ]*
M3: Build	Output / results display	Jayden	11/10	Output code based on input testing	*[ ]*
M4: Testing	Create test circuits and verify results	Jayden	11/17	Test file and results	*[ ]*
M5: Final	README and final documentation	T	11/24	README.md	*[ ]*
M5: Final	Final presentation	All	12/08/26	Presentation and live demo	*[ ]*

Status key: Not Started / In Progress / Complete

4.2 Communication

The team communicates through an Instagram group chat, and members are expected to respond within 24 hours. The team meets once a week to review progress and plan the next week's tasks.

4.3 Progress Tracking

Progress is tracked with GitHub Issues, one per task. Each member works on their own feature branch and opens a pull request, and another member reviews it before it is merged into main.

GitHub Actions builds and tests every pull request. Each weekly meeting starts with a status check on every open task.

4.4 Addressing Challenges

A member who is stuck or behind notifies the group chat within 24 hours so others can help. If a task is still behind after a week, it is split up or reassigned at the next meeting. Disagreements are discussed as a team and decided by majority vote.

5. Contribution Table

Team Member	Completed This Iteration	Responsibilities for Next Iteration
Jayden Tran	Drafted sections 1 and 6	Define the netlist input format and start building the parser that reads circuits from a text file or the terminal. Create the first set of hand-solved test circuits.
Joseph Gereige	Drafted sections 4 and 2, helped fill in timeline	Write the method summary for nodal analysis and Gaussian elimination, and start building the solver engine.
Tianhong Zhong	Drafted section 3, filled in timeline, set up github repository	Set up README.md, set up data structures

6. Team Meetings and Decisions

6.1 Meeting Log

Date	Attendees	Topics Discussed	Decisions Made	Action Items
10/13/26	*Jayden T, Tianhong Z, Joseph G*	*Iteration #1*	*Choose our roles, goals, and making our own deadlines.*	*Shown above*
TBD	*[Names]*	*[Topics]*	*[Decisions]*	*[Who does what]*
TBD	*[Names]*	*[Topics]*	*[Decisions]*	*[Who does what]*

6.2 Summary of Key Decisions

During this iteration, the team agreed to build Circuit Solver as a C++ console program for DC circuits containing resistors and independent voltage and current sources. AC analysis, reactive components, and dependent sources were left out so that the project can be finished and tested well within the semester. Nodal analysis with Gaussian elimination was chosen as the solving method because it works for any circuit of this type and turns directly into a matrix problem that can be programmed. The work was divided into three parts, input parsing and validation, the solver engine, and output and testing, so that each member owns one part of the program, and all three can work at the same time once the shared data structures are defined. The team set its target as matching hand-calculated answers within 0.1% on at least 10 test circuits, which gives a clear and measurable standard for when the solver is correct. For workflow, the team decided to use feature branches with peer-reviewed pull requests on GitHub, weekly progress meetings, and an Instagram group chat with a 24-hour response expectation. This keeps every member aware of what the others are working on and makes problems visible early.